Scientific Calculator

Software Production Engineering (SPE) Mini Project Report

Parag Piprewar

MT2025083

March 7, 2026

IIIT Bangalore

Github Repo: https://github.com/ParagRider1/scientific-calculator-devops

1. Introduction

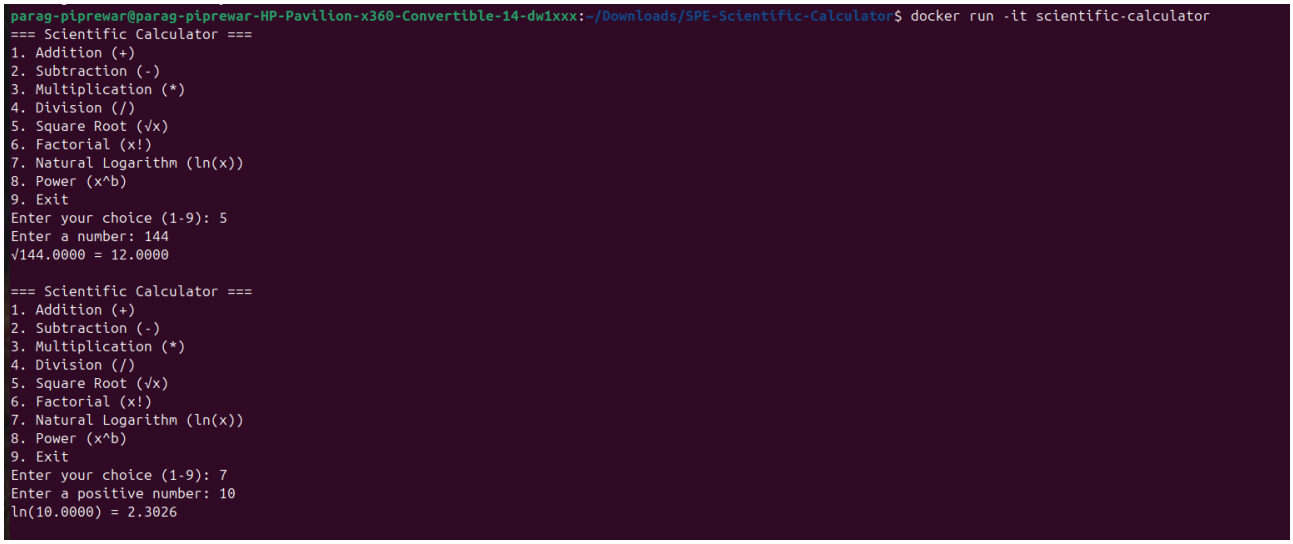
Modern software development requires automation of building, testing, and deployment processes. DevOps practices help teams integrate development and operations through automation tools and continuous integration pipelines.

This project demonstrates the implementation of a **DevOps toolchain** for a Java-based scientific calculator application. The goal of the project is to automate the process of building, testing, containerizing, and deploying the application using a CI/CD pipeline.

The project integrates multiple DevOps tools including GitHub, Jenkins, Maven, JUnit, Docker, DockerHub, and Ansible to form a complete automated pipeline.

The application itself is a **Scientific Calculator** that supports mathematical operations such as square root, factorial, natural logarithm, and power functions.

Fig: Scientific Calculator application



```
parag-piprewar@parag-piprewar-HP-Pavilion-x360-Convertible-14-dw1xxx:~/Downloads/SPE-Scientific-Calculator$ docker run -it scientific-calculator
=== Scientific Calculator ===
1. Addition (+)
2. Subtraction (-)
3. Multiplication (*)
4. Division (/)
5. Square Root (√x)
6. Factorial (x!)
7. Natural Logarithm (ln(x))
8. Power (x^b)
9. Exit
Enter your choice (1-9): 5
Enter a number: 144
√144.0000 = 12.0000

=== Scientific Calculator ===
1. Addition (+)
2. Subtraction (-)
3. Multiplication (*)
4. Division (/)
5. Square Root (√x)
6. Factorial (x!)
7. Natural Logarithm (ln(x))
8. Power (x^b)
9. Exit
Enter your choice (1-9): 7
Enter a positive number: 10
ln(10.0000) = 2.3026
```

---

2. Objectives of the Project

The main objectives of this project are:

- Implement a Java based scientific calculator.
- Use version control for managing source code.

- Automate testing using JUnit.
- Build the project using Maven.
- Implement continuous integration using Jenkins.
- Containerize the application using Docker.
- Store Docker images in DockerHub.
- Deploy the containerized application using Ansible.

This project demonstrates how DevOps practices enable faster and reliable software delivery.

---

3. DevOps Toolchain Used

The following tools were used in the project pipeline:

Tool	Purpose
GitHub	Source code version control
Jenkins	Continuous Integration
Maven	Build automation
JUnit	Automated testing
Docker	Containerization
DockerHub	Docker image repository
Ansible	Automated deployment

These tools work together to automate the entire lifecycle of the application.

4. Application Description

The implemented application is a **Scientific Calculator** written in Java. The application allows users to perform common mathematical operations through a command line interface.

Supported Operation

- Addition
- Subtraction
- Multiplication
- Division
- Square Root
- Factorial
- Natural Logarithm
- Power Function

The calculator provides a menu-driven interface allowing users to select operations and input values interactively.

Example Execution

```
=== Scientific Calculator ===
1. Addition (+)
2. Subtraction (-)
3. Multiplication (*)
4. Division (/)
5. Square Root (√x)
6. Factorial (x!)
7. Natural Logarithm (ln(x))
8. Power (x^b)
9. Exit
```

The application is packaged as a **JAR file using Maven**, which can be executed directly or inside a Docker container.

(Screenshot Placeholder – Application Execution)

---

5. DevOps Pipeline Overview

The DevOps pipeline implemented in this project automates the entire software lifecycle.

Pipeline flow:

```
GitHub → Jenkins → Maven Build → JUnit Testing → Docker Image → DockerHub → Ansible Deployment
```

Whenever code is pushed to the GitHub repository, Jenkins automatically triggers the pipeline through a webhook. The pipeline then performs the following steps:

1. Clone the source code
2. Build the application
3. Run automated tests
4. Build the Docker image
5. Push the image to DockerHub
6. Deploy the container using Ansible

This automation ensures that the application is continuously tested and deployed without manual intervention.

---

6. Jenkins Pipeline Implementation

The core automation logic of the project is implemented in the **Jenkinsfile**.

The Jenkins pipeline consists of multiple stages that execute sequentially.

Jenkinsfile Structure

```
pipeline {
 agent any
 stages {
   Clone Git
   Build
   Test
   Verify JAR
   Build and Push Docker Image
   Deploy with Ansible
 }
}
```

Each stage performs a specific step in the CI/CD pipeline.

ENVIRONMENT VARIABLES

environment {

DOCKER_IMAGE_NAME = 'scientific-calculator'

GITHUB_REPO_URL = 'https://github.com/ParagRider1/scientific-calculator-devops.git'

DOCKER_HUB_USERNAME = 'paragrider1'

```
}
```

---

7. Jenkins Pipeline Stages Explanation

7.1 Clone Git Stage

This stage retrieves the latest source code from the GitHub repository.

Purpose:

- Fetch updated code
- Ensure pipeline always runs on latest version

Example command used internally:

```
stages {
    stage('Clone Git') {
      steps {
        script {
          git branch: 'main',
              credentialsId: 'github_credentials',
              url: "${GITHUB_REPO_URL}"
        }
      }
    }
```

7.2 Build Stage

The build stage compiles the Java source code and packages the application using Maven.

Command executed:

```
mvn clean package -DskipTests
```

Explanation:

- `clean` removes previous build artifacts
- `package` compiles the code and generates a JAR file

Output:

```
target/ScientificCalculator-1.0-SNAPSHOT.jar
```

7.3 Test Stage

This stage runs automated tests using **JUnit**.

Command executed:

```
mvn test
```

Categories of Test Cases

The test suite contains multiple test cases that validate different mathematical operations and edge cases.

Square Root Tests

The square root tests verify behavior for:

positive numbers, zero, decimal numbers, negative inputs

Factorial Tests

Factorial calculations are tested for:

zero, one, positive integers, larger numbers

Natural Logarithm Tests

Logarithm test cases verify behavior for:

value 1, mathematical constant *e*, positive numbers, zero, negative inputs

Power Function Tests

Power calculations are tested using different base and exponent combinations.

Test scenarios include:

positive exponents, negative exponents, fractional exponents, negative bases

Edge Case Testing

Several tests check extreme or special conditions such as:

division by zero, infinity results, NaN values, very large numbers

Example:

```
@Test
void testDivisionByZero() {
    double result = 5.0 / 0.0;
    assertTrue(Double.isInfinite(result));
}
```

Test Results

Example result:

```
Tests run: 47
Failures: 0
Errors: 0
```

Observation:

All test cases passed successfully, confirming that the application logic works correctly.

7.4 Verify JAR Stage

This stage verifies whether the build process successfully generated the application JAR file.

Command executed:

```
ls -lh target/
```

Purpose:

- Confirm the JAR file exists
- Validate successful build output

Example result:

```
ScientificCalculator-1.0-SNAPSHOT.jar
```

7.5 Build and Push Docker Image Stage

In this stage the application is containerized using Docker.

The Dockerfile defines the environment required to run the Java application.

Example Dockerfile logic:

```
FROM eclipse-temurin:17
WORKDIR /app
COPY target/ScientificCalculator-1.0-SNAPSHOT.jar app.jar
CMD ["java","-jar","app.jar"]
```

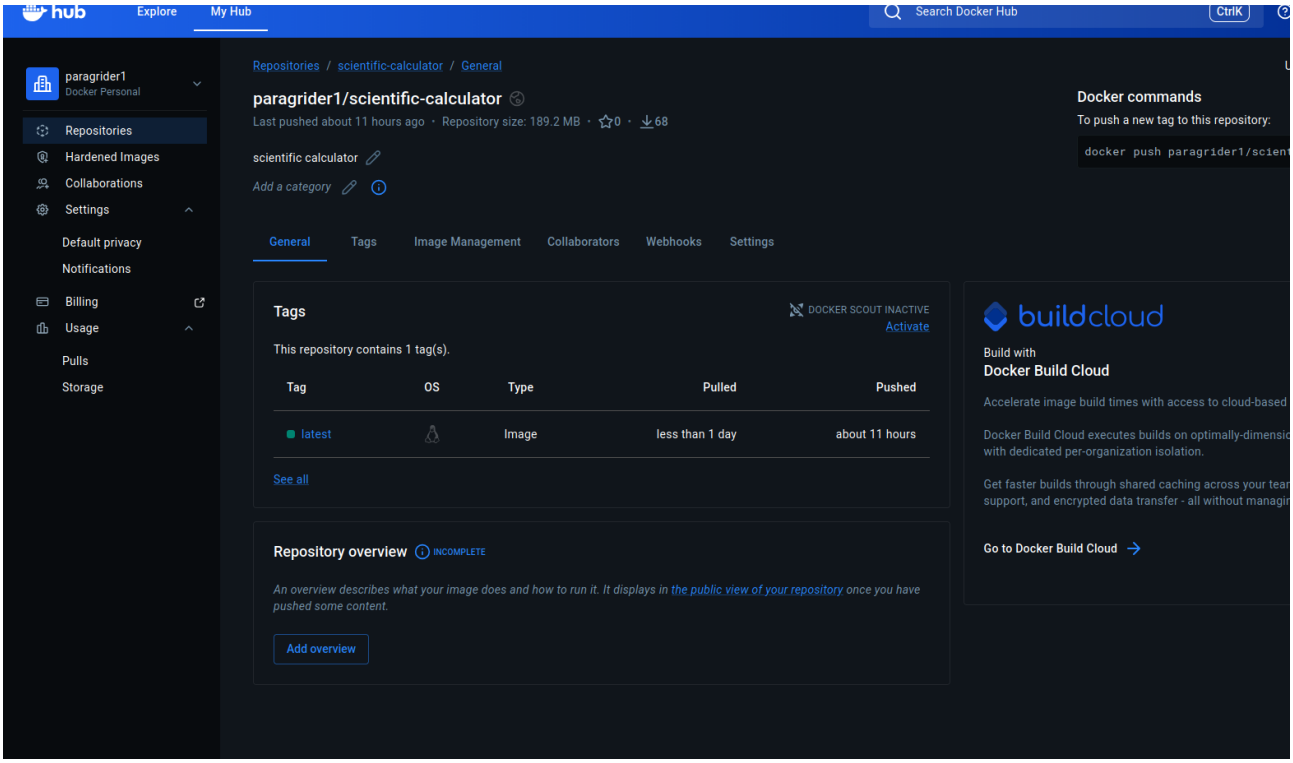
The pipeline builds the Docker image and pushes it to DockerHub.

Commands executed:

```
docker build -t username/scientific-calculator .
docker push username/scientific-calculator
```

Purpose:

- Create portable container image
- Store image in DockerHub repository



7.6 Deployment with Ansible

The final stage deploys the application using **Ansible**.

The Ansible playbook performs the following tasks:

1. Pull the Docker image from DockerHub
2. Stop any existing container
3. Start a new container

Example deployment command:

```
ansible-playbook -i inventory.ini deploy.yml
```

Observation:

The deployment process automatically launches the container containing the scientific calculator application.

Playbook structure:

```
- hosts: servers
  tasks:
    - name: Pull Docker image
      command: docker pull paragrider1/scientific-calculator
    - name: Stop old container if running
      command: docker stop scientific-calculator
```

```
    ignore_errors: yes
  - name: Remove old container
    command: docker rm scientific-calculator
    ignore_errors: yes
  - name: Run new container
    command: docker run -d --name scientific-calculator paragrider1/scientific-calculator
```

---

8. Execution of the Application

The containerized calculator application can be executed using Docker.

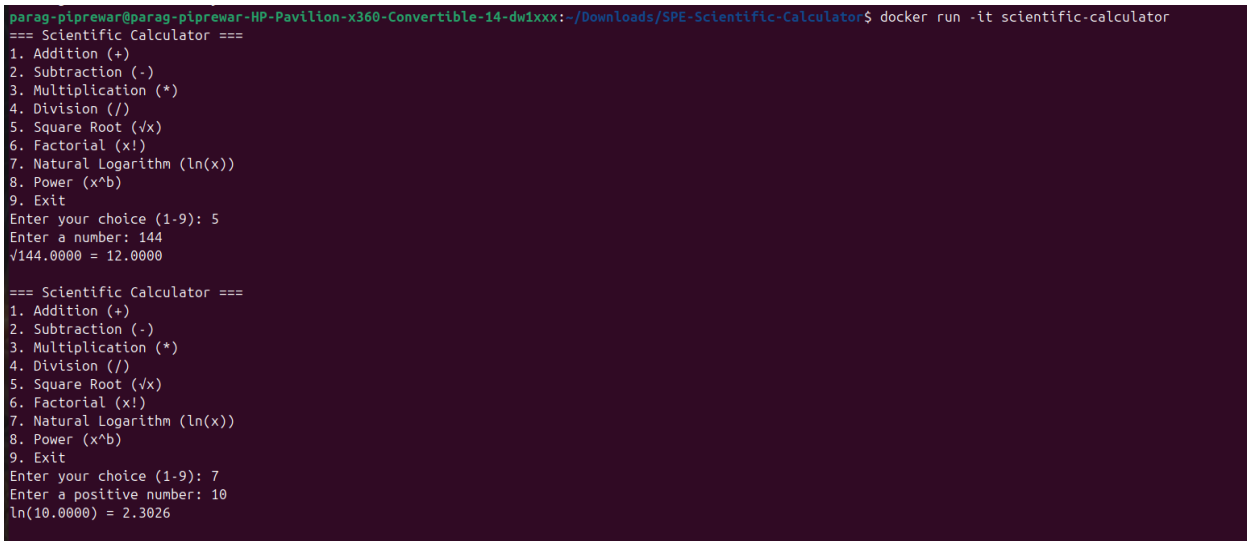
Example command:

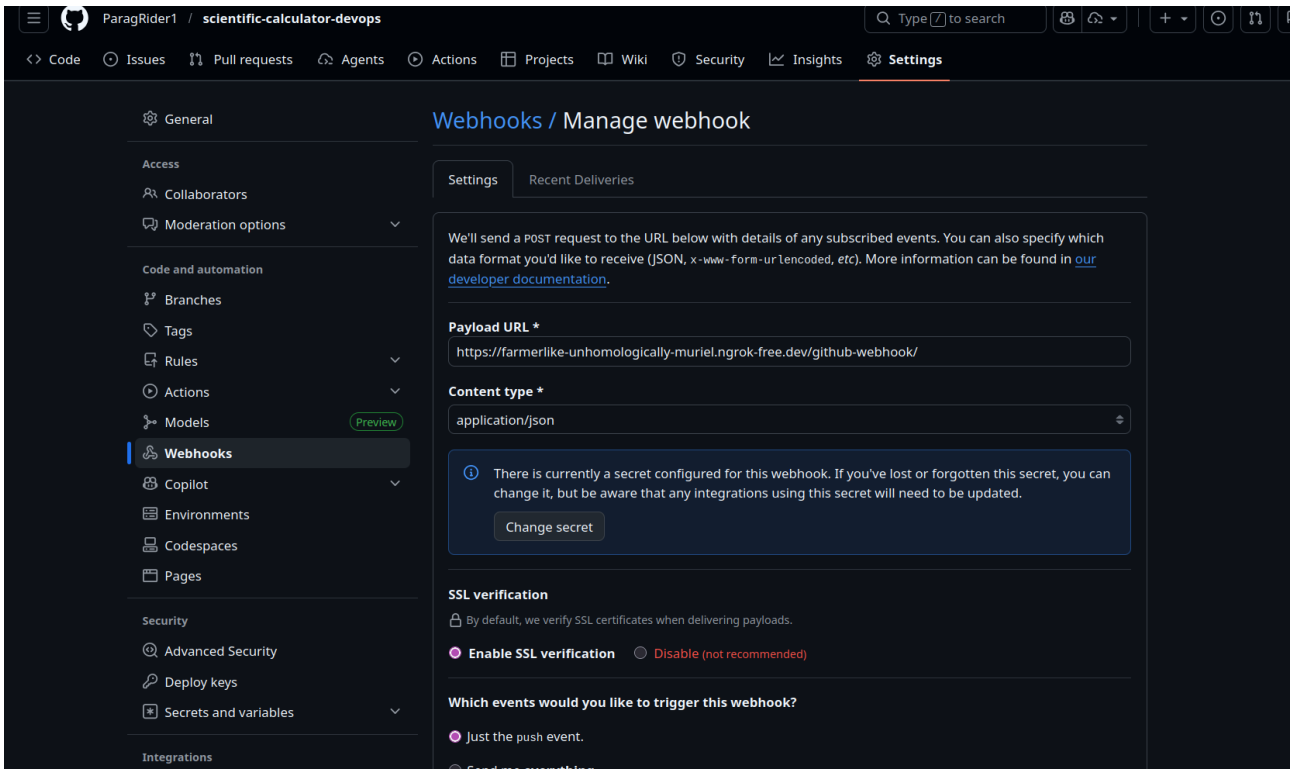
```
docker run -it scientific-calculator
```

This launches the calculator inside a Docker container and allows users to interact with it through the terminal.

Alternatively, the application can also be executed directly using the generated JAR file.

```
java -jar target/ScientificCalculator-1.0-SNAPSHOT.jar
```





Webhook configuration

Email notification on build status

---

9. Observations

Several important observations were made during the implementation of the DevOps pipeline:

- Jenkins automation significantly reduces manual work in building and testing applications.
- Maven provides a structured and repeatable build process.
- Docker ensures that the application runs consistently across different environments.
- DockerHub enables easy sharing and distribution of container images.

- Ansible simplifies deployment by automating container execution.

Overall, the DevOps pipeline improves reliability and efficiency in software development workflows.

---

10. Conclusion

This project successfully demonstrates the implementation of a complete DevOps pipeline for a Java based scientific calculator application.

By integrating tools such as GitHub, Jenkins, Maven, JUnit, Docker, DockerHub, and Ansible, the project automates the entire lifecycle of the application from source code to deployment.

The CI/CD pipeline ensures that every code change is automatically built, tested, containerized, and deployed, which reflects real-world DevOps practices used in modern software development.

The implementation highlights how DevOps tools can streamline development processes and enable faster, more reliable software delivery.